

(This is a cleaned-up extract from a 2002 essay called "[Free software UI](#)"; I cut it down to only the discussion of preferences, fixed some dead links, and removed some clunky adverbs, but it's true to the original and still written relative to 2002.)

CHOOSING OUR PREFERENCES

The most-often-mentioned example of bad free software UI is “too many preferences.” This is pretty much my pet peeve; I was inspired to write a [whole window manager](#) on the premise that current window managers suffer from preferences overload.

A traditional free software application is configurable so that it has the union of all features anyone's ever seen in any equivalent application on any other historical platform. Or even configurable to be the union of all *applications* that anyone's ever seen on any historical platform (Emacs *cough*).

Does this hurt anything? Yes it does. It turns out that preferences have a cost. Of course, *some* preferences also have important benefits - and can be crucial interface features. But each one has a price, and you have to carefully consider its value. Many users and developers don't understand this, and end up with a lot of cost and little value for their preferences dollar.

Too many preferences means you can't find any of them.

While I use X-Chat every day and am fundamentally friendly to it, I have to pick on it in this paragraph and hope the maintainer is good-natured. It has so many preferences that there's a whole menu just to hold all the different preferences dialogs. Plus you can script it - in your choice of languages. If I want to change something about X-Chat, it takes me eons just to find the right button.

Preferences substantively damage QA and testing.

As someone who reads dozens of bug reports per day, and occasionally fixes a couple, I can tell you that it's extremely common to find a bug that only happens if some certain combination of options are enabled. As a software developer, I can tell you that preferences frequently do add quite a bit of code complexity as well; some more than others, but even simple preferences can add a lot of complexity if there are 200 of them.

Upshot: more preferences means fewer real features, and more bugs.

Preferences make integration and good UI difficult.

One of the hardest lessons of GUI programming is that hard coding behavior can be the Right Thing. Programmers are taught to make everything generic and infinitely flexible. The problem is that the more generic and infinitely flexible your UI is, the more similar it is to a programming language. Lisp is not a good user interface.

The point of a good program is to *do something specific and do it well*.

The thing is that each UI decision depends on countless other UI decisions. A simple example is keybindings. On UNIX/Linux, it's nearly impossible to pick reasonable default bindings for global desktop navigation because they all conflict with bindings that some app is using. On Windows, the desktop navigation bindings are hardcoded, and no app uses them, because apps know for sure which bindings to avoid.

If your program has no idea what the rest of its environment (or even all the parts of itself) will be like, then it can't have reasonable defaults. It can't “just work.” At some point the existence of preferences means that everyone *must* configure their desktop; while ideally preferences are *optional* and the default behavior is reasonable without intervention. Hard coding what can be hard coded has a lot of value.

Preferences keep people from fixing real bugs.

One of the more amusing functions in GNU Emacs is “menu-bar-enable-clipboard.” Now that KDE is fixed, Emacs is the last remaining X application that insists on [having cut and paste that doesn't work correctly](#). So they have this function “menu-bar-enable-clipboard” which means “please make my cut and paste work correctly.” Why is this an option? I call this kind of preference the “unbreak my application please” button. Just fix the app and be done with it.

(2015 update: Emacs finally fixed this in Emacs 24, in 2012.)

Preferences can confuse many users.

Take the famous “too many clocks” example *(2015 note - this doesn't seem to be online anymore, but it was a usability study Sun did in 2001)*. A significant number of test subjects were so surprised to have 5 choices of clock they couldn't figure out how to add a clock to their panel. This cost of preferences is invariably underestimated by us technical types.

Finite pixels.

The simplest issue: *the preferences dialog is finite in size*. You can see many large apps struggling with where to put the things. Mozilla has a zillion prefs that aren't even in the dialog.

SO HOW DO YOU DECIDE WHICH PREFERENCES TO HAVE?

On hearing that preferences have a cost, some people get upset that we're going to remove every preference, or that their favorite one is gone, or whatever.

Step back for a minute and think about the preferences problem.

For any program, there are literally an infinite number of possible preferences. Each one has a cost. A program with infinite preferences is therefore infinitely bad. But clearly some preferences are genuinely useful and important. So the UI developer's job is to choose the useful subset of possible preferences.

An argument that “preferences are good” or “preferences are bad” is clearly unproductive. Only an argument that draws a line between when a preference should exist, and when it should not, is a meaningful argument that impacts real-world developer decisions.

The traditional, de facto free software line between when a preference should exist and when it shouldn't is “a preference should exist if someone implements it or asks for it.” No one is going to seriously defend this one though. At least, no one who's maintained an application and seen the sheer number of non-overlapping feature/preference requests you typically receive can take this one seriously.

Just as Linus rejects most kernel patches these days - but probably took more of them back in the Linux 0.1 era - as free software matures, more preferences will be rejected.

So how is the decision made? It's a judgment call. I try to go through some questions like these:

- Ask questions to find out what's bugging someone who requests a preference. What is the annoyance or inefficiency that prompted them to ask?
- Can said annoyance be made to go away for all users without requiring a preference? If so, just do that. You may have to think about the big picture of the UI to figure out how to make it Just Work.
- Is the annoyance or inefficiency significant, or did it cost them 1 second doing something that users do once per week on average? If it's just some trivial thing, then the extra feature or preference probably costs more than it's worth, even if you can't make things Just Work.
- Does any other OS have this feature or preference? I'm all for innovation, but if no one else is doing something, you should think it through twice to be sure there isn't a *reason* they aren't doing it. If you're appropriately humble you'll probably find that a lot of thought and user testing has gone into the currently popular platforms.

The main point is to set limits. Figure you have some fixed number of slots for preferences; is the preference in question worth “spending” one of those slots? Once spent it's hard to free a slot up again. Keep in mind that preferences have a cost, and demand that each preference have some real value.

STANDING UP TO USER PRESSURE

As the maintainer of a free software package, it's tough to stand up to the continuous barrage of feature requests, many of which are “requests for preferences,” often accompanied by patches.

Reading dozens of GNOME and Red Hat bugs per day, I find that users ask for a preference by default. If a user is using my app FooBar and they come to something they think is stupid - say the app deletes all their email - it's extremely common that they'll file a bug saying “there should be an option to disable eating all my email” instead of one saying “your craptastic junk-heap of an app ate my email.” People just assume that FooBar was designed to eat your email, and humbly ask that you let them turn off this feature they don't like.

Fight the temptation! Admit the truth to your users - you're a loser, FooBar just sucks and ate their email. ;-) This feature should be fixed, not made optional.

I got a patch just today because the Metacity window shading animation is lame-looking, and moreover grabs the X server which results in keeping all apps from repainting during the animation. So the patch made the animation optional. But that's probably not the right fix. As far as I currently know, the problem is that the animation sucks. Based on that info, the preference would be a band-aid.

Linus had a good quote on this, found in [LWN](#) this week. The argument on the linux-kernel list was apparently about whether some poorly-implemented IDE feature should have been removed prior to creating a new, better replacement implementation. Linus says:

The fact is, many things are easier to fix afterwards. Particularly because that's the only time you'll find people motivated enough to bother about it. If you were to need to fix everything before-the-fact, nothing fundamental would ever get fixed, simply because the people who can fix one thing are not usually the same people who can fix another.

The annoying problem here is that sometimes you have to temporarily cause regressions in order to get things fixed. For GNOME 2 for example, we wanted to fix the window list (task bar) applet; under GNOME 1.4, it has to be manually sized via preferences. We wanted it to Just Work. But it took a while to make it Just Work and we were flamed regularly during its period of brokenness, because we didn't get it quite right. Most of those flames demanded re-adding all the manual sizing preferences. But that would have been wrong. (Now that the applet is mostly working, it may even make sense to add a “maximum size” preference as a useful feature.)

Stick to your guns and get it right.

WHAT ABOUT ADVANCED USERS?

[As Joel points out](#) advanced users do not want a million useless preferences when things should just work on their own. The point rings true in my experience.

Sometimes there may be a good reason for an Advanced tab or the like, but something as elaborate as the Nautilus 1 “user levels” concept isn’t warranted IMO, and Advanced tabs must not be used as band-aids or excuses for stupid designs.

(The Desktop Preferences->Advanced menu in current GNOME 2 betas is precisely such a band-aid, because the new control panels haven’t fully replaced the functionality of the old ones; this menu needs to die.)

WHY ALL THIS OBSESSION WITH PREFERENCES ANYWAY?

I find that if you’re hard-core disciplined about having good defaults that Just Work instead of lazily adding preferences, that naturally leads the overall UI in the right direction. Issues come up via bugzilla or mailing lists or user testing, and you fix them in some way other than adding a preference, and this means you have to think about the right UI and the right way to fix problems.

Using preferences as a band-aid is the root of much UI evil. It’s a big deal and a big change in direction for free software that GNOME has figured this out.